Relatório: Folhagem & Cia

Candidato: Pedro Wellington **Branch:** `desafio/pedro-wellington`

1. O que foi feito

Implementei a API da loja de plantas com as 7 rotas pedidas:

Rota	Status de sucesso
`POST /plantas`	201
`GET /plantas` (filtros `?luminosidade=` e `?petFriendly=`, juntos ou separados)	200
`GET /plantas/:id`	200
`PATCH /plantas/:id`	200
`DELETE /plantas/:id`	204
`POST /plantas/:id/venda`	200
`GET /plantas/estoque/valor-total`	200

Todas as regras de negócio foram implementadas: campos obrigatórios no cadastro, `preco > 0`, `regaACadaDias >= 1`, `estoque >= 0`, `luminosidade` restrita ao enum, 400 para dados inválidos, 404 para id inexistente e venda que nunca deixa o estoque negativo.

Bônus feitos:

- Busca por nome sem diferenciar maiúsculas: `?nome=costela`
- Ordenação: `?ordem=preco` ou `?ordem=nome` (qualquer outro valor retorna 400)
- Rota `GET /plantas/esgotadas`

2. Como foi feito

Segui a ordem sugerida no desafio, com um commit por etapa:

1. **Model e migration:** enum `Luminosidade` e model `Planta` no `schema.prisma`, com a migration `create_planta_model`.
2. **Repository:** `PlantaRepository.ts`, a única camada que fala com o Prisma.
3. **Controller:** `PlantaController.ts`, com validação, regras de negócio e `try/catch` que responde 500 em erro inesperado.
4. **Rotas:** registradas em `routes.ts`, com as rotas fixas (`/plantas/estoque/valor-total` e `/plantas/esgotadas`) antes de `/plantas/:id`, para o Express não tratar "esgotadas" como id.
5. **Filtros, venda e valor total.**
6. **Testes** no Insomnia, seguindo a tabela da parte 4 do enunciado.
7. **Bônus.**

Decisões e o porquê:

- **Validação com `=== undefined`** em vez de `!campo`, porque `!0` e `!false` são verdadeiros e recusariam um estoque zero ou um `petFriendly: false`, que são válidos.
- **Uma única função de validação** (`validarCampos`) usada no cadastro e na edição: no POST todos os campos são obrigatórios, no PATCH só os enviados são validados, mas com as mesmas regras.
- `Number.isInteger` para preço, estoque e quantidade, o que recusa `"8990"`, `89.9` e `NaN`.
- **Buscar antes de editar, deletar ou vender**, para devolver 404 em vez de deixar o erro do Prisma cair no `catch` e virar 500.
- **Só os campos da planta vão ao banco**, nunca o `req.body` inteiro.
- **Filtros não enviados ficam `undefined`**, e o Prisma ignora esses campos no `where`, então uma única consulta cobre todas as combinações.
- **Preço em centavos (inteiro)**, como pede o enunciado, para evitar erros de arredondamento com decimais.
- **Valor total calculado no código** (`reduce` sobre `preco × estoque`), buscando só esses dois campos com `select`.
- **Não houve decisão diferente do enunciado**, exceto a forma de rodar as migrations (seção 3).

3. Principais dificuldades

- **Docker não abria:** o Docker Desktop mostrava "Virtualization is disabled in firmware". Confirmei no Gerenciador de Tarefas (Virtualização: Desabilitado) e ativei o **SVM Mode** na BIOS.
- **`yarn migration` não alcançava o banco (P1001):** o container do banco não publica a porta para o Windows, então o comando, que roda no meu PC, não conseguia conectar. Para não alterar o `docker-compose.yml`, passei a rodar as migrations dentro do container da API: `docker compose exec server npx prisma migrate dev`.

4. Uso de IA

Usei o **Claude** durante o desafio.

Para quê:

- Entender e resolver os erros de ambiente listados acima (Docker, BIOS, portas, migrations, Prisma Client).
- Entender partes de código e saber o que deve ser usado em cada situação.
- Aprender conceitos: o que é uma API, para que serve o Docker, o fluxo rota → controller → repository → Prisma → banco, e o que significam os comandos usados.
- Usei para formatar o meu relatório para uma melhor visualização no modo markdown.
- Usei para gerar os dados de exemplo das plantas (nome, espécie, preço, estoque, luminosidade etc.).

Como conferi:

- Testei no Insomnia os casos da tabela da parte 4 do enunciado (cadastro válido e inválido, busca, edição, filtros, venda, valor total, deleção) e os bônus, verificando o status esperado em cada um.
- Conferi a conta do valor total na mão.
- Rodei `yarn typecheck`, que passou sem erros.
- Li o código comparando com o `UserController` e o `UserRepository`.
- Sempre conferi se a recomendação que a IA mandava seguia o mesmo modelo do código base (o UserController e o UserRepository) e testei no Insomnia antes de dar como certo. O que aprendi é que

preciso sempre verificar o que a IA diz, porque nem tudo que ela responde está certo: algumas orientações não bateram, e só percebi isso rodando, lendo o erro e testando.